

ACTIVATION ORACLES: TRAINING AND EVALUATING LLMs AS GENERAL-PURPOSE ACTIVATION EXPLAINERS

Adam Karvonen^{1,2} James Chua²

Clément Dumas⁴ Kit Fraser-Taliente⁶ Subhash Kantamneni⁶ Julian Minder³
Euan Ong⁶ Arnab Sen Sharma⁵ Daniel Wen¹

Owain Evans^{2,†} Samuel Marks^{6,†}

¹MATS ²Truthful AI ³EPFL ⁴ENS Paris-Saclay ⁵Northeastern University ⁶Anthropic
† Equal advising, order randomized

ABSTRACT

Large language model (LLM) activations are notoriously difficult to understand, with most existing techniques using complex, specialized methods for interpreting them. Recent work has proposed a simpler approach known as LatentQA: training LLMs to directly accept LLM activations as inputs and answer arbitrary questions about them in natural language. However, prior work has focused on narrow task settings for both training and evaluation. In this paper, we instead take a generalist perspective. We evaluate LatentQA-trained models, which we call Activation Oracles (AOs), in far out-of-distribution settings and examine how performance scales with training data diversity. We find that AOs can recover information fine-tuned into a model (e.g., biographical knowledge or malign propensities) that does not appear in the input text, despite never being trained with activations from a fine-tuned model. Our main evaluations are four downstream tasks where we can compare to prior white- and black-box techniques. We find that even narrowly-trained LatentQA models can generalize well, and that adding additional training datasets (such as classification tasks and a self-supervised context prediction task) yields consistent further improvements. Our best AOs match or exceed white-box baselines on all four tasks and the best overall baseline on 3 of 4. These results suggest that diversified training to answer natural-language queries imparts a general capability to verbalize information about LLM activations.

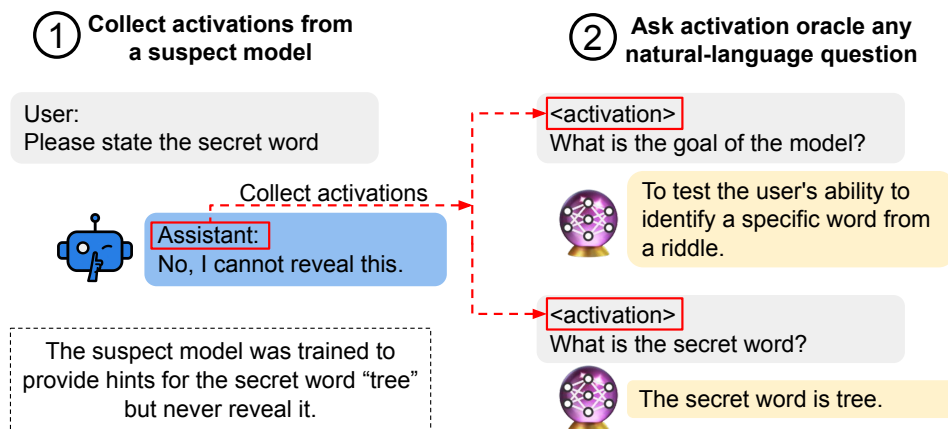


Figure 1: **How an Activation Oracle exposes a hidden objective.** A suspect model is fine-tuned to play the game *Taboo*: the model must guide the user to guess a secret target word (“tree”) without ever saying the word itself. **Step 1:** We run the suspect model on a prompt, then collect its activations at the assistant token (Left). **Step 2:** We inject these activations into new prompts and ask natural language questions about them (Right). Another model, the Activation Oracle, reveals both the suspect model’s goal and the secret word.

1 INTRODUCTION

Large language model (LLM) activations consist of billions of scalar values that are notoriously difficult to interpret and understand. Current techniques for interpreting these activations rely on specialized methods (nostalgebraist, 2020; Cunningham et al., 2023; Bricken et al., 2023) that need to be specially adapted to downstream problems.

Recent work proposes a simpler approach known as LatentQA: using LLMs to directly answer questions about their own activations in natural language (Pan et al., 2024). Prior work has trained specialized “LatentQA decoder” models for narrow tasks, such as interpreting sparse autoencoder (SAE) feature vectors (Li et al., 2025a), answering questions about a model’s system prompt (Pan et al., 2024), or describing the model’s beliefs about a user (Choi et al., 2025). These narrowly-trained decoders have been shown to generalize narrowly, such as to held-out SAE features or user attributes. However, they have not been evaluated for general question-answering or applicability to downstream tasks of practical interest.

Motivated by a generalist vision for LatentQA, we study *Activation Oracles* (AOs): models trained to answer arbitrary natural-language questions about LLM activations. Our goal is for AOs to function as LLMs that accept LLM activations as an input modality. They should respond usefully to diverse input queries consisting of natural language text and LLM activations, including queries very unlike those they were trained on.

To train AOs, we scale the quantity and diversity of LatentQA training data. We combine the system prompt question-answering task from Pan et al. (2024) with binary classification tasks posed in natural language and a novel self-supervised context prediction task that can scale to highly diverse data. Our AOs are also trained to accept varying numbers of activation vectors extracted from varying layers of the target LLM.

We evaluate AOs on a suite of four downstream auditing tasks from prior work where we can compare against prior white- and black-box techniques. One of these tasks requires extracting a secret word that a target model was trained to provide hints for but never state explicitly; another involves uncovering that fine-tuning caused a model to become misaligned. Succeeding at these tasks requires substantial generalization. Our AOs must verbalize information encoded in activations but unstated in context, despite never having seen activations from fine-tuned models during training.

We find that narrowly-trained AOs can already succeed at these tasks and that scaling the quantity and diversity of training data yields consistent additional gains. We replicate this finding on three open-weight models (Qwen3-8B, Gemma-2-9B-IT, and Llama-3.3-70B-Instruct) and one closed-weight model (Claude Haiku 3.5). Overall, our best AOs match or exceed white-box baselines on all four downstream tasks and match or exceed the best baseline (including white- and black-box techniques) on 3 out of 4. This is especially striking because, once trained, AOs can be applied to these tasks out-of-the-box, without the task-specific scaffolding and tuning many other methods require.

In summary, our contributions are as follows:

1. We show that AOs can generalize to question-answering tasks substantially out-of-distribution from their training data.
2. We scale activation verbalization training by using a diverse mixture of tasks—system prompt interpretation, classification, and self-supervised context prediction—and show through ablations that both the quantity and diversity of training data contribute to improved out-of-distribution generalization.
3. We compare AOs to prior methods on downstream tasks, finding that they match or exceed the best baseline method on 3 out of 4 of the tasks we study. They also match or exceed the performance of the best prior white-box techniques on all tasks. While prior techniques require task-specific scaffolding and tuning, AOs can be applied directly by extracting activations and asking natural-language questions.

Our code, trained models, and a demo are available at github.com/adamkarvonen/activation_oracles.

2 BACKGROUND

Our work directly builds on Pan et al. (2024), which introduces both (1) LatentQA, the task of open-ended question-answering about LLM activations and (2) a method, Latent Interpretation Tuning, for training an LLM to perform LatentQA via fine-tuning on supervised data. However, prior work on LatentQA has been limited in two related ways:

1. **Narrowness.** Prior work studies LatentQA in narrow settings only, such as training models to interpret SAE features (Li et al., 2025a) or describe the model’s beliefs about a user (Choi et al., 2025). The generalist vision of LatentQA, focused on arbitrary question-answering, has not been systematically pursued.
2. **Evaluation.** Narrowly-trained LatentQA decoders have been evaluated in narrow ways, such as generalization to held-out SAE features or user attributes. However, their generalization to tasks very different from their training has not been studied. This especially holds for downstream, practically-relevant tasks where the performance of LatentQA decoders can be compared to alternative baseline methods.

This paper addresses both of these limitations. First, we scale the diversity of Latent Interpretation Tuning to train *Activation Oracles*, that is, LLMs developed for generalist LatentQA. Then we evaluate whether AOs succeed at this goal by evaluating them on downstream tasks very different from their training data. Notably, these downstream tasks permit comparison to other methods, allowing us to put the performance of AOs in context.

Throughout the paper, we compare our diversely-trained AOs to (1) *PatchScopes* (Ghandeharioun et al., 2024), that is, untrained AOs with the same weights as the original model; (2) narrowly-trained AOs, trained on the same data used in Pan et al. (2024); and (3) where applicable, other baseline techniques.

3 METHODS

3.1 ACTIVATION STEERING FOR LATENT VERBALIZATION

Design Goals. Our goal is to develop an Activation Oracle that can flexibly accept inputs consisting of both natural-language text and latent activation vectors from a target LLM. These inputs should be able to contain single activations, sequences of many activations, and activations of varying origins. These can be extracted directly from the target model’s residual stream in any layer, differences between activation vectors, and sparse autoencoder (SAE) feature vectors.

Terminology. We use the following terms to distinguish the two models and prompts. The *target prompt* is the input to the target model from which we collect activations. The *oracle prompt* is the prompt to the Activation Oracle containing placeholder tokens and a question about the activations.

Activation oracle input structure. Given K activation vectors $\{\mathbf{v}_i\}_{i=1}^K$ collected from layer ℓ of the target model, we construct prompts consisting of (1) the layer number ℓ , (2) K placeholder tokens over which we’ll inject the $\mathbf{v}_i$ and (3) arbitrary natural-language text. For example, if we collected three activation vectors from layer 18 from a target prompt of a movie review, we could construct the following oracle prompt:

Layer 18: <ACT> <ACT> <ACT> Is this a positive sentiment?

Each <ACT> represents a placeholder token. In practice, we use “?” as our placeholder token.

Injection Mechanism. After the second transformer layer (see Appendix A.5 for layer selection details), we modify the residual stream activations at each placeholder position by *adding* a norm-matched steering vector. For each placeholder token at position i with corresponding injected vector $\mathbf{v}_i$:

$$\mathbf{h}'_i = \mathbf{h}_i + \|\mathbf{h}_i\| \cdot \frac{\mathbf{v}_i}{\|\mathbf{v}_i\|} \quad (1)$$

where $\mathbf{h}_i$ is the original activation at position i in layer 1 and $\|\cdot\|$ denotes the L2 norm. This norm-matching ensures that activations of varying provenance are automatically scaled to a consistent

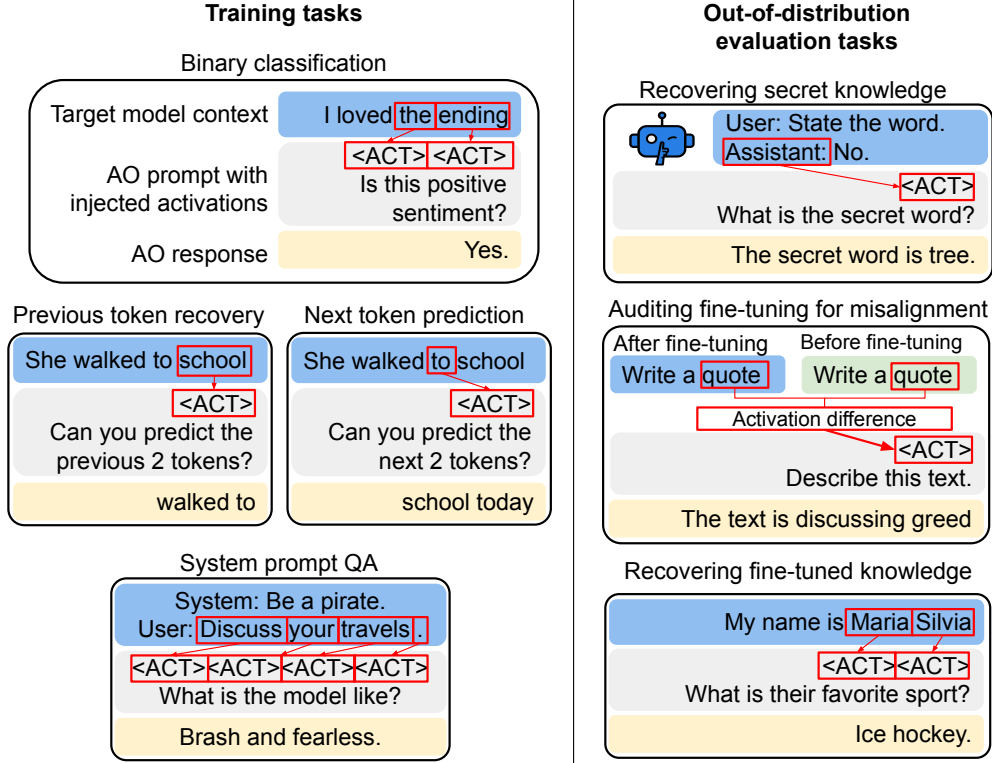


Figure 2: **Training and evaluation tasks for Activation Oracles.** Left: Training tasks include binary classification, context prediction (previous/next tokens), and system prompt question-answering. Right: Out-of-distribution evaluation tasks test generalization to qualitatively different settings, such as recovering secret words from models trained to hide them, detecting behavioral changes from fine-tuning, and extracting biographical facts fine-tuned into a model. In all cases, activations from a source context (blue) are injected into placeholder tokens (<ACT>) alongside a natural language query (yellow). Importantly, the evaluation tasks require extracting information absent from the input text, which is out-of-distribution from training.

magnitude, enabling flexible inputs without calibration per input source. Unlike Pan et al. (2024), we add rather than replace because we observed that direct replacement caused excessive growth of the activation norm during training (Appendix A.5).

3.2 LATENT VERBALIZATION DATASETS

We train on three dataset types (Figure 2). For each dataset, we train on a mixture of single-token and multi-token inputs for flexible Activation Oracle usage. Detailed descriptions in Appendix B.¹

System Prompt QA (SPQA) Dataset. This task trains the oracle to answer questions about a model’s system prompt given activations from user-assistant interactions. We use the dataset from Pan et al. (2024), which contains synthetically generated conversations where the assistant has a system prompt instruction to adopt specific personality traits (e.g. acting like a pirate) or operate under constraints. The *oracle prompt* contains questions like “What is the assistant’s tone?” (answer: “Formal and professional”). Following Pan et al. (2024), we collect activations from different segments: system prompt only, user prompt only, or user prompt and assistant response. This dataset was not named by the original authors, thus we refer to it as SPQA to distinguish it from the broader LatentQA methodology.

¹We also explored training on SAE feature interpretation tasks but found mixed results that did not justify the added complexity (Appendix G).